

AWS Production Workloads — Complete Interview Answer Guide

Real-Time Scenarios for Every Service

How to Answer This Question

Interviewer asks:

"Tell me about your experience operating production workloads on AWS with Elastic Beanstalk, EKS, EC2, RDS, VPC, IAM, S3, CloudWatch, and Secrets Manager."

Wrong answer  — just listing services:

"I have used EC2, RDS, S3, and CloudWatch in my projects."

Right answer  — real scenario with problem + solution + result:

"In production, I ran a Spring Boot microservices platform on EKS serving 2 million requests/day. Let me walk you through how each service played a specific role in that architecture..."

The Master Scenario — Set This Up First

"I worked on a B2B SaaS platform — a financial transaction processing system handling 2 million API requests per day, 99.99% uptime SLA, PCI-DSS compliance, and peak loads during market hours (9am-4pm EST). Let me walk you through how each AWS service was used in production and the real problems I solved."

Architecture Overview:

Internet



Route 53 (DNS)



CloudFront (CDN)



Application Load Balancer



├─ EKS Cluster (Microservices – Spring Boot)



├─ Payment Service (3 pods)



├─ User Service (2 pods)



├─ Order Service (4 pods)



└─ Notification (2 pods)



├─ Elastic Beanstalk (Admin Portal – monolith)



└─ EC2 (Legacy Batch Processing Jobs)



VPC (private subnets)



├─ RDS PostgreSQL (Multi-AZ)



├─ ElastiCache Redis



└─ S3 (documents, reports, backups)

Cross-cutting: IAM, CloudWatch, Secrets Manager, KMS

1 VPC — The Foundation Everything Runs On

Real-Time Scenario

"Before deploying any service, I designed the VPC architecture. Early on, a developer accidentally exposed our RDS database to the internet with a misconfigured security group. No breach occurred, but it triggered an internal audit. After that, I redesigned the entire network with strict subnet isolation."

VPC: 10.0.0.0/16 (us-east-1)

```
|
|— Public Subnets    (10.0.1.0/24, 10.0.2.0/24) ← AZ-1a, AZ-1b
|   └─ NAT Gateway, Load Balancer, Bastion Host
|
|— Private Subnets   (10.0.3.0/24, 10.0.4.0/24) ← AZ-1a, AZ-1b
|   └─ EKS Worker Nodes, EC2, Elastic Beanstalk
|
|— Data Subnets      (10.0.5.0/24, 10.0.6.0/24) ← AZ-1a, AZ-1b
|   └─ RDS, ElastiCache (NO internet access ever)
```

Key decisions I made:

- ✓ 3-tier subnet model (public, private, data)
- ✓ NAT Gateway in each AZ (no single point of failure)
- ✓ VPC Flow Logs enabled → sent to CloudWatch for audit
- ✓ Private endpoints for S3 and Secrets Manager
(traffic never leaves AWS network)
- ✓ Security Groups as virtual firewalls per service
(RDS only accepts from app-sg, not from internet)

Security Group Design

bash

Application Security Group – accepts only from ALB

```
aws ec2 create-security-group \  
  --group-name app-sg \  
  --description "App tier – accepts from ALB only"
```

Allow only ALB to reach app servers (port 8080)

```
aws ec2 authorize-security-group-ingress \  
  --group-id sg-app-id \  
  --protocol tcp --port 8080 \  
  --source-group sg-alb-id    # ← only ALB, not 0.0.0.0/0!
```

Database Security Group – accepts only from app tier

```
aws ec2 authorize-security-group-ingress \  
  --group-id sg-db-id \  
  --protocol tcp --port 5432 \  
  --source-group sg-app-id    # ← only app servers, not internet!
```

Interview Answer — VPC

"VPC was the foundation of everything. I designed a 3-tier network: public subnets for load balancers and NAT gateways, private subnets for application servers, and isolated data subnets for RDS and ElastiCache with no internet route whatsoever. After a security audit flagged an exposed database, I implemented VPC Flow Logs fed into CloudWatch and created a policy preventing any security group from opening port 5432 to 0.0.0.0/0 using AWS Config rules. That caught two more misconfigurations automatically before they reached prod."

2 IAM — Security and Access Control

Real-Time Scenario

"We had an incident where a developer's leaked AWS access key caused \$14,000 in unexpected EC2 charges overnight — someone spun up GPU instances for crypto mining. That forced us to completely overhaul our IAM strategy."

IAM Strategy After the Incident:

Before ❌	After ✅
Long-lived access keys	No access keys at all
Admin privileges for devs	Least-privilege roles
No MFA enforced	MFA mandatory for all
Single shared account	Separate dev/staging/prod accounts
No key rotation policy	Automated rotation + alerts
No CloudTrail	CloudTrail + alerting on root usage

Real IAM Implementation

json

```

// IAM Role for EKS Payment Service Pod (IRSA)
// Only this pod can access these specific resources
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "secretsmanager:GetSecretValue"
      ],
      "Resource": [
        "arn:aws:secretsmanager:us-east-1:123456:secret:prod/payment/db-creds*",
        "arn:aws:secretsmanager:us-east-1:123456:secret:prod/payment/stripe-key*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::payment-receipts-prod/*"
    },
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "Bool": { "aws:MultiFactorAuthPresent": "false" }
      }
    }
  ]
}

```

yaml

```
# IRSA – IAM Roles for Service Accounts (EKS best practice)
# Each microservice gets its own IAM role – no shared credentials!
apiVersion: v1
kind: ServiceAccount
metadata:
  name: payment-service-sa
  namespace: production
  annotations:
    # This SA assumes exactly this IAM role – nothing more
    eks.amazonaws.com/role-arn: arn:aws:iam::123456:role/payment-service-role
```

Interview Answer — IAM

"After a credential leak cost us \$14,000, I moved to a zero long-lived credentials model. On EKS, I implemented IRSA — each microservice gets its own Service Account mapped to a scoped IAM role. No access keys anywhere. For developers, all access is through SSO with MFA — temporary credentials via AWS CLI STS assume-role, lasting 1 hour. I also added a CloudWatch alarm on CloudTrail that fires a PagerDuty alert within 60 seconds if the root account is used or if any IAM policy grants admin to a new principal. Zero incidents in 18 months after that."

3 EC2 — Compute for Batch and Legacy Workloads

Real-Time Scenario

"We had a nightly batch job that processed 500,000 financial transaction records — reconciling payments between our system and banks. It ran on a fixed EC2 instance. One night the instance ran out of disk at 2am. I woke up to 200 failed transactions and a very unhappy finance team."

Before ❌

After ✅

Single EC2, always running
Fixed disk (20GB EBS)
No monitoring
SSH for deployments
Manual AMI updates
No backup

Spot Instance Fleet + On-Demand fallback
EFS mount (auto-expanding)
CloudWatch disk + CPU alarms
Systems Manager Session Manager (no SSH)
Auto Scaling Group + Launch Template
AWS Backup daily snapshots

EC2 Implementation — Launch Template

```
bash

# Create Launch Template for batch workers
aws ec2 create-launch-template \
  --launch-template-name batch-worker-lt \
  --version-description "v1" \
  --launch-template-data '{
    "ImageId": "ami-0abcdef1234567890",
    "InstanceType": "m5.xlarge",
    "IamInstanceProfile": { "Name": "batch-worker-profile" },
    "SecurityGroupIds": ["sg-batch-id"],
    "UserData": "base64-encoded-startup-script",
    "BlockDeviceMappings": [{
      "DeviceName": "/dev/xvda",
      "Ebs": {
        "VolumeSize": 100,
        "VolumeType": "gp3",
        "DeleteOnTermination": true,
        "Encrypted": true
      }
    }],
    "MetadataOptions": {
      "HttpTokens": "required"
    }
  }'

# Use Spot with On-Demand fallback — 70% cost saving!
aws ec2 request-spot-fleet \
  --spot-fleet-request-config '{
    "AllocationStrategy": "lowestPrice",
    "TargetCapacity": 3,
    "SpotPrice": "0.15",
    "LaunchSpecifications": [...],
    "OnDemandTargetCapacity": 1
  }'
```

Interview Answer — EC2

"We used EC2 for our legacy batch processing — it wasn't worth re-architecting to EKS yet. After a disk-full incident at 2am caused 200 failed transactions, I redesigned the setup: moved to a Spot Instance Fleet with one On-Demand fallback for reliability, mounted EFS instead of EBS so disk never ran out, replaced SSH access with Systems Manager Session Manager for

full audit trail, and added CloudWatch alarms for disk above 70%, CPU above 90%, and batch job duration exceeding 4 hours. Cost dropped 68% with Spot Instances and zero incidents in 14 months."

Elastic Beanstalk — The Admin Portal

Real-Time Scenario

"Our admin portal was a Spring Boot monolith built by a team that didn't have deep Kubernetes knowledge. Elastic Beanstalk was the right choice — managed deployments, auto-scaling, and health checks without the team needing to manage infrastructure."

Elastic Beanstalk Setup:

Environment: production-admin

Platform: Java 17 running on 64-bit Amazon Linux 2023

Load Balancer: Application (ALB)

Auto Scaling: Min 2, Max 6 instances (t3.medium)

Deployment: Rolling with additional batch (zero downtime)

Health Check: /actuator/health (enhanced health reporting)

yaml

```
# .ebextensions/01-jvm-options.config
option_settings:
  aws:elasticbeanstalk:application:environment:
    SERVER_PORT: 5000
    SPRING_PROFILES_ACTIVE: production
    JAVA_TOOL_OPTIONS: "-Xmx512m -Xms256m"

  aws:elasticbeanstalk:environment:process:default:
    HealthCheckPath: /actuator/health
    HealthCheckInterval: 30

  aws:autoscaling:asg:
    MinSize: 2
    MaxSize: 6

  aws:autoscaling:trigger:
    MeasureName: CPUUtilization
    Statistic: Average
    Unit: Percent
    UpperThreshold: 70
    LowerThreshold: 30
    UpperBreachScaleIncrement: 2
    LowerBreachScaleIncrement: -1

  aws:elasticbeanstalk:command:
    DeploymentPolicy: RollingWithAdditionalBatch
    BatchSizeType: Percentage
    BatchSize: 30          # update 30% of instances at a time
    Timeout: 600
```

Interview Answer — Elastic Beanstalk

"We used Elastic Beanstalk for our admin portal — a Spring Boot monolith maintained by a team that wasn't Kubernetes-native. Beanstalk handled load balancing, auto-scaling, and deployments automatically. I configured RollingWithAdditionalBatch deployment policy — it spins up new instances before terminating old ones, so we had zero downtime deployments even with just 2 instances. I used .ebextensions to inject environment variables from Secrets Manager at startup rather than hardcoding them. The ops overhead was near zero — the team just ran eb deploy and Beanstalk handled the rest."

5 EKS — Microservices Orchestration

Real-Time Scenario

"Our payment service started dropping requests during market open at 9:30am EST every day. We had 3 pods running but load spiked to 10x in 90 seconds. By the time HPA scaled new pods (90 seconds), thousands of requests had timed out. Users were reporting failed payments on social media."

The Problem:

9:30am spike: 10x traffic in 90 seconds

HPA scale-up time: 90 seconds

Result: ~4,500 timed-out requests per day

The Solution — 3 changes:

1. Pre-warm pods before 9:30am using CronJob
2. Reduce HPA scale-up stabilization from 5min to 30sec
3. Add Cluster Autoscaler for node-level scaling

yaml

```

# HPA – tuned for financial spike patterns
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: payment-service-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: payment-service
  minReplicas: 5          # raised from 3 to handle morning spike
  maxReplicas: 30
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60  # lowered from 80 – scale earlier!
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 30  # was 300 – too slow!
    policies:
      - type: Pods
        value: 5                      # add 5 pods at once
        periodSeconds: 30
    scaleDown:
      stabilizationWindowSeconds: 300 # scale down slowly – avoid thrash

---

# CronJob – pre-warm payment pods before market open
apiVersion: batch/v1
kind: CronJob
metadata:
  name: pre-warm-payment
spec:
  schedule: "25 9 * * 1-5"  # 9:25am weekdays – 5 min before open
  jobTemplate:
    spec:
      template:
        spec:
          containers:

```

```
- name: scaler
  image: bitnami/kubectl
  command:
    - kubectl
    - scale
    - deployment/payment-service
    - --replicas=15
    - -n production
```

```
# PodDisruptionBudget – never kill all pods at once
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: payment-pdb
spec:
  minAvailable: 3      # always keep 3 pods during upgrades
  selector:
    matchLabels:
      app: payment-service
```

bash

```
# IRSA setup – payment service gets its own IAM role
eksctl create iamserviceaccount \
  --name payment-service-sa \
  --namespace production \
  --cluster prod-cluster \
  --role-name payment-service-role \
  --attach-policy-arn arn:aws:iam::123456:policy/payment-service-policy \
  --approve

# Node groups – separate for different workloads
eksctl create nodegroup \
  --cluster prod-cluster \
  --name payment-nodes \
  --node-type m5.2xlarge \
  --nodes-min 3 \
  --nodes-max 15 \
  --node-labels workload=payment \
  --taints dedicated=payment:NoSchedule # only payment pods here
```

"EKS ran our 8 microservices in production. The hardest problem was the daily 9:30am payment spike — 10x traffic in 90 seconds and HPA couldn't scale fast enough. I solved it three ways: raised minimum replicas from 3 to 5, reduced HPA stabilization window from 5 minutes to 30 seconds, and added a CronJob that pre-scaled payment pods to 15 at 9:25am every weekday — before the spike hit. Failed payments dropped from 4,500/day to zero. I also used IRSA so each microservice had its own scoped IAM role — no shared credentials between services."

6 RDS — Database in Production

Real-Time Scenario

"At 11pm on a Friday, our RDS PostgreSQL CPU hit 100% and the application went down. I got paged. Root cause was a single unindexed query doing a full table scan on 80 million rows — a developer had pushed it that afternoon without review."

Production RDS Setup:

Engine: PostgreSQL 15
Instance: db.r6g.2xlarge (Multi-AZ)
Storage: 1TB gp3, auto-scaling to 2TB
Backup: Daily automated, 30-day retention
Encryption: KMS at rest + in transit
Read Replica: 1 replica in same region for read traffic
Subnet: Data subnets only (no internet access)
Parameter Group: Custom (shared_buffers, work_mem tuned)

bash

```

# The incident – bad query causing full table scan
# This query was scanning 80 million rows every 30 seconds!
SELECT * FROM transactions
WHERE EXTRACT(MONTH FROM created_at) = 3
AND status = 'PENDING';
# No index on created_at expression – full table scan ❌

# Fix 1: Add index (without downtime using CONCURRENTLY)
CREATE INDEX CONCURRENTLY idx_txn_month_status
ON transactions (DATE_TRUNC('month', created_at), status)
WHERE status = 'PENDING';

# Fix 2: Enable Performance Insights in RDS Console
# → immediately showed this query as the top consumer

# Fix 3: Rewrite query to use index
SELECT * FROM transactions
WHERE created_at >= '2024-03-01'
AND created_at < '2024-04-01'
AND status = 'PENDING';
# Now uses index – 80ms instead of 45 seconds ✅

# Fix 4: Set statement timeout to kill runaway queries
-- In RDS parameter group:
statement_timeout = 30000 -- 30 seconds max per query

# Fix 5: Read replica for reporting queries
# Redirect heavy analytical queries to replica
spring:
  datasource:
    write-url: jdbc:postgresql://rds-writer.endpoint:5432/db
    read-url:  jdbc:postgresql://rds-reader.endpoint:5432/db

```

Interview Answer — RDS

"RDS PostgreSQL was our primary database — Multi-AZ for failover, one read replica for analytics queries, encrypted with KMS. The most memorable incident was a Friday night outage from an unindexed query doing a full table scan on 80 million rows — CPU hit 100% and the app went down in 3 minutes. I used RDS Performance Insights to identify the exact query in under 2 minutes, added a CONCURRENTLY index (no table lock), rewrote the query, and set statement_timeout to 30 seconds to kill future runaway queries. Resolution time was 22 minutes. After that, I added a pre-deployment SQL review step in our CI pipeline using pganalyze to catch missing indexes before they reach production."

7 S3 — Object Storage

Real-Time Scenario

"We stored financial reports and transaction receipts in S3 — 8TB of data growing at 500GB/month. A compliance audit required us to prove that no report was deleted or tampered with for 7 years. We also had a data breach scare when we discovered a bucket was accidentally set to public."

S3 Setup for Compliance:

Bucket: payment-documents-prod

- ✓ Versioning enabled (keeps every version forever)
- ✓ Object Lock — COMPLIANCE mode, 7 years
(even AWS support cannot delete!)
- ✓ Block ALL public access — account level
- ✓ SSE-KMS encryption (customer-managed key)
- ✓ Access Logs → separate logging bucket
- ✓ Replication → cross-region backup (us-west-2)
- ✓ Lifecycle rules:
 - 0-30 days → S3 Standard
 - 30-90 days → S3 Standard-IA (40% cheaper)
 - 90-365 days → S3 Glacier (68% cheaper)
 - 365+ days → S3 Glacier Deep Archive (95% cheaper)

json

```
// S3 Bucket Policy – deny any non-HTTPS access
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::payment-documents-prod",
        "arn:aws:s3:::payment-documents-prod/*"
      ],
      "Condition": {
        "Bool": { "aws:SecureTransport": "false" }
      }
    },
    {
      "Effect": "Deny",
      "Principal": "*",
      "Action": ["s3:DeleteObject", "s3:DeleteObjectVersion"],
      "Resource": "arn:aws:s3:::payment-documents-prod/*"
    }
  ]
}
```

bash

```
# Generate pre-signed URL – temporary access without credentials
# Expires in 1 hour – user downloads their receipt without
# needing AWS credentials or making bucket public
aws s3 presign s3://payment-documents-prod/receipts/TXN-12345.pdf \
  --expires-in 3600

# Returns: https://payment-documents-prod.s3.amazonaws.com/...?X-Amz-Signature=...
# Share this URL with user – it expires automatically
```

Interview Answer — S3

"S3 stored all financial documents — receipts, reports, statements. 8TB growing at 500GB/month. A compliance audit required 7-year immutability. I enabled S3 Object Lock in COMPLIANCE mode — even admins cannot delete objects during the retention period. Added lifecycle rules to automatically tier data from Standard to Glacier after 90 days, saving 68% on storage costs. When a public bucket incident was discovered, I immediately applied

Account-level Block Public Access — it overrides any bucket-level permission. For user downloads, I used pre-signed URLs expiring in 1 hour instead of making anything public. The audit passed with zero findings."

8 CloudWatch — Monitoring and Alerting

Real-Time Scenario

"We had a memory leak in our order service that built up slowly over 3 days — not enough to trigger an alarm but enough to cause gradual degradation. By day 3, orders were taking 8 seconds to process instead of 200ms. Users were complaining but no alarm had fired. We were flying blind."

CloudWatch Implementation After the Incident:

1. Custom Metrics — pushed from application:
 - order_processing_duration_ms (histogram)
 - payment_success_rate (gauge)
 - active_db_connections (gauge)
 - jvm_heap_used_bytes (gauge)
2. Log Insights Queries — automated anomaly detection:
 - P99 latency trending UP over 6 hours → alert
 - Error rate > 0.1% over 5 min → PagerDuty
 - Memory growing > 5% per hour → warning
3. Dashboards — one per service + one executive summary
4. Composite Alarms — alert only when MULTIPLE signals fire (reduces false positives by 80%)

java

```
// Push custom metrics from Spring Boot to CloudWatch
@Component
@Slf4j
public class PaymentMetricsPublisher {

    private final CloudWatchClient cloudWatch;
    private final String NAMESPACE = "Production/PaymentService";

    public void recordPaymentDuration(long durationMs, String status) {
        cloudWatch.putMetricData(r -> r
            .namespace(NAMESPACE)
            .metricData(
                MetricDatum.builder()
                    .metricName("PaymentDurationMs")
                    .value((double) durationMs)
                    .unit(StandardUnit.MILLISECONDS)
                    .dimensions(
                        Dimension.builder()
                            .name("Status").value(status).build(),
                        Dimension.builder()
                            .name("Environment").value("prod").build()
                    )
                    .timestamp(Instant.now())
                    .build()
            )
        );
    }
}
```

bash

```
# CloudWatch Logs Insights – find the memory leak retrospectively
fields @timestamp, @message, jvm_heap_used_bytes
| filter service = "order-service"
| stats avg(jvm_heap_used_bytes) as avg_heap by bin(1h)
| sort @timestamp asc
# → showed 12% growth per hour over 72 hours – memory leak confirmed

# Create Alarm: P99 latency > 2 seconds for 5 minutes
aws cloudwatch put-metric-alarm \
  --alarm-name "OrderService-P99-Latency-High" \
  --alarm-description "P99 latency above 2s" \
  --metric-name order_processing_p99_ms \
  --namespace Production/OrderService \
  --statistic p99 \
  --period 300 \
  --threshold 2000 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 1 \
  --alarm-actions arn:aws:sns:us-east-1:123456:pagerduty-critical

# Composite Alarm – fire only if BOTH CPU high AND error rate high
# Prevents waking someone at 2am for a single metric spike
aws cloudwatch put-composite-alarm \
  --alarm-name "OrderService-Critical" \
  --alarm-rule "ALARM(OrderService-CPU-High) AND ALARM(OrderService-ErrorRate-High)"
```

Interview Answer — CloudWatch

"After a 3-day memory leak went undetected because we only had infrastructure metrics, I implemented application-level custom metrics from Spring Boot — JVM heap, P99 latency, payment success rate, active DB connections. I used CloudWatch Logs Insights to run hourly anomaly detection — if P99 latency trended up more than 20% over 6 hours, it fired a warning before users felt it. I also switched to Composite Alarms that only page on-call if BOTH CPU and error rate are elevated — reduced false-positive pages by 80%. After the changes, our MTTD (mean time to detect) dropped from 3 days to 11 minutes."

9 Secrets Manager — Credential Management

Real-Time Scenario

"A developer committed a database password to a public GitHub repo. We caught it in 4

minutes via GitGuardian, but the 4 minutes was 4 minutes too long. That forced an emergency credential rotation at 11pm. After that, I eliminated all hardcoded credentials from every service."

Before ❌

After ✅

DB password in application.yml	Secrets Manager – auto-rotation
API keys in .env files	Secrets Manager – versioned
Secrets in K8s ConfigMaps	External Secrets Operator → Secrets Manager
Manual rotation (quarterly)	Automatic rotation (Lambda, 30 days)
No audit of who accessed what	CloudTrail logs every GetSecretValue call

java

```
// Spring Boot – fetch secret from Secrets Manager at startup
// No credentials in any config file!
@Configuration
public class DatabaseConfig {

    @Value("${SECRET_ARN}")
    private String secretArn;

    @Bean
    public DataSource dataSource() {
        // Fetch secret – rotates automatically, app always gets latest
        SecretsManagerClient client = SecretsManagerClient.create();

        String secretJson = client.getSecretValue(r ->
            r.secretId(secretArn)).secretString();

        ObjectMapper mapper = new ObjectMapper();
        JsonNode secret = mapper.readTree(secretJson);

        return DataSourceBuilder.create()
            .url(secret.get("url").asText())
            .username(secret.get("username").asText())
            .password(secret.get("password").asText()) // no hardcoding!
            .build();
    }
}
```

yaml

```
# EKS – External Secrets Operator syncs Secrets Manager → K8s Secret
# The actual secret never lives in your Git repo or K8s etcd unencrypted
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: payment-db-secret
  namespace: production
spec:
  refreshInterval: 1h           # re-sync every hour (picks up rotations)
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: payment-db-creds      # creates this K8s Secret
  data:
    - secretKey: db-password
      remoteRef:
        key: prod/payment/db-credentials
        property: password
    - secretKey: stripe-api-key
      remoteRef:
        key: prod/payment/stripe
        property: api_key
```

```
bash

# Automatic rotation setup – Lambda rotates every 30 days
aws secretsmanager rotate-secret \
  --secret-id prod/payment/db-credentials \
  --rotation-lambda-arn arn:aws:lambda:us-east-1:123456:function:rotate-rds-secret \
  --rotation-rules AutomaticallyAfterDays=30

# Audit: who accessed this secret in the last 24 hours?
aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=ResourceName,AttributeValue=prod/payment/db-credentials \
  --start-time 2024-01-01T00:00:00 \
  --end-time 2024-01-02T00:00:00
```

Interview Answer — Secrets Manager

"After a DB password was committed to GitHub and we had an emergency credential rotation at 11pm, I moved every secret to Secrets Manager. No credentials in application.yml, .env files,

or K8s ConfigMaps. On EKS, I used External Secrets Operator to sync Secrets Manager values into Kubernetes Secrets — the operator polls every hour so when a secret rotates automatically, pods pick it up without restart. For RDS, I set up automatic rotation every 30 days via a Lambda function. Every GetSecretValue call is logged to CloudTrail — we can audit exactly who or what accessed a credential and when. Zero credential incidents in 20 months after that."

Final — One Paragraph Master Answer

"I operated a production financial platform on AWS processing 2 million requests per day. I designed the VPC with 3-tier subnet isolation — public for load balancers, private for application servers, data-only subnets for RDS with zero internet access. IAM used IRSA on EKS giving each microservice its own scoped role with no long-lived credentials — after a key leak cost us \$14,000. EKS ran 8 microservices with tuned HPA and pre-warming CronJobs to handle daily 10x traffic spikes at market open. Elastic Beanstalk hosted our admin monolith with zero-downtime rolling deployments. RDS PostgreSQL ran Multi-AZ with a read replica and Performance Insights that helped me resolve a CPU crisis in 22 minutes. S3 stored compliance documents with Object Lock for 7-year immutability and lifecycle rules that cut storage costs by 65%. CloudWatch custom metrics from Spring Boot cut our mean time to detect incidents from 3 days to 11 minutes. Secrets Manager with External Secrets Operator eliminated all hardcoded credentials — every secret rotates automatically every 30 days. Each service taught me something the hard way, and each incident made the system more resilient."

Quick Interview Reference — One Line Per Service

Service	Your One-Line Answer
VPC	"3-tier subnet isolation — public LB, private app, data-only RDS. VPC Flow Logs to CloudWatch caught 2 misconfigs automatically."
IAM	"Zero long-lived credentials. IRSA on EKS, SSO+MFA for humans. Alert fires in 60 seconds if root account is used."
EC2	"Batch jobs on Spot Fleet with On-Demand fallback. 68% cost reduction. Systems Manager replaced SSH for full audit trail."
Elastic Beanstalk	"Admin portal, RollingWithAdditionalBatch deployment, zero downtime. .ebextensions pulled secrets from Secrets Manager."

Service	Your One-Line Answer
EKS	"8 microservices, IRSA per service, tuned HPA with pre-warming CronJob eliminated 4,500 daily payment failures."
RDS	"Multi-AZ PostgreSQL, read replica for analytics, Performance Insights found a runaway query in 2 minutes."
S3	"Object Lock COMPLIANCE mode for 7-year immutability, lifecycle rules saved 65% storage cost, pre-signed URLs for secure access."
CloudWatch	"Custom app metrics from Spring Boot, Composite Alarms cut false pages by 80%, MTTD dropped from 3 days to 11 minutes."
Secrets Manager	"External Secrets Operator syncs to EKS, auto-rotation every 30 days, CloudTrail audits every access. Zero credential incidents."

AWS Production Experience — Senior Engineer Interview Preparation